

Algoritmen en computationeel denken

27 oefeningen op papier, met een antwoordblad achteraan.

Naam _____ Datum _____

- Schrijf met potlood, dan kan je gerust iets uitgommen en opnieuw proberen.
- Bij een rekenvraag: schrijf eerst de bewerking op en zet de eenheid erbij.
- Bij een keuze: zeg niet alleen wát je kiest, maar ook waaróm.
- Het antwoordblad zit achteraan. Scheur het eraf voor je begint.

1 Wat is een algoritme?

1. Wat is een algoritme? _____
2. Algoritme of niet?
 - a) een recept om een cake te bakken _____
 - b) een stappenplan om een boterham te smeren _____
 - c) de code van een computerspel _____
3. Een algoritme moet altijd op een computer draaien.
☐ waar ☐ niet waar
4. Wat is het verschil tussen een digitaal en een niet-digitaal algoritme? Geef van elk een voorbeeld.

5. In een algoritme mag je de stappen zomaar van volgorde wisselen.
☐ waar ☐ niet waar

6. Wat gebeurt er als één stap in een algoritme ontbreekt?

2 Een algoritme ontwerpen

7. Hoe heet een schema met pijlen dat de stappen van een algoritme toont?

8. Waarvoor dient een flowchart?

9. Waarvoor staat IPO bij het analyseren van een probleem?

10. Noem drie stappen die je volgt bij het ontwerpen van een algoritme.

11. In welke programmeertaal vertaal je volgens de vakfiche je algoritme? _____

12. Waarom test je een algoritme uit, en wat doe je als het niet het juiste resultaat geeft?

13. Wat is de laatste stap bij het ontwerpen van een algoritme?

3 Computationeel denken

14. Welk principe?

- a) details weglaten die er niet toe doen _____
- b) een probleem in kleinere delen opsplitsen _____
- c) gelijkenissen tussen problemen zien _____

15. Welke vier principes van computationeel denken noemt de vakfiche?

16. Abstractie betekent zoveel mogelijk details toevoegen.

- ☐ waar ☐ niet waar

17. Je schrijft een algoritme om de weg naar school uit te leggen. Noem een detail dat je weglaat, en zeg welk principe dat is.

18. Een spel heeft tien niveaus die sterk op elkaar lijken. Welk principe helpt hier, en wat win je ermee?

19. Noem drie momenten waarop decompositie helpt.

20. Wat is een voorbeeld van decompositie bij het maken van een spel?

4 Fouten en efficiëntie

21. Hoe heet een fout in een programma? _____
22. Hoe heet het opsporen en oplossen van zo'n fout? _____
23. Een programma moet een vierkant tekenen maar maakt maar drie zijden. Wat doe je?

24. Een fout in een algoritme vind je alleen door ernaar te kijken, nooit door het uit te voeren.

☐ waar ☐ niet waar

25. Efficiënt betekent dat een algoritme zoveel mogelijk stappen heeft.

☐ waar ☐ niet waar

26. Waarom is een korter algoritme met dezelfde uitkomst meestal beter?

27. Noem drie dingen die je helpen om een efficiënt algoritme te schrijven.

Antwoorden

Dit blad is voor wie meekijkt. Scheur het eraf voor je de bundel geeft.

1. Wat is een algoritme?

- 1** een stappenplan met instructies in een vaste volgorde
- 2** a) een algoritme · b) een algoritme · c) een algoritme
- 3** niet waar
- 4** Een niet-digitaal algoritme voer je zonder computer uit, bijvoorbeeld een stappenplan om een boterham met choco te smeren. Een digitaal algoritme laat je door een computer uitvoeren, bijvoorbeeld een lampje laten flikkeren met een programma.
- 5** niet waar
- 6** Het resultaat klopt meestal niet meer.

2. Een algoritme ontwerpen

- 7** een flowchart of stroomdiagram
- 8** Om de oplossing visueel voor te stellen, zodat je in één blik ziet hoe de stappen op elkaar volgen.
- 9** input, process en output
- 10** Je formuleert de probleemstelling, je analyseert het probleem met IPO, en je test het algoritme uit.
- 11** Scratch
- 12** Je test het om te zien of het echt doet wat je wilde. Geeft het niet het juiste resultaat, dan stuur je het bij.
- 13** testen en bijsturen waar nodig

3. Computationeel denken

- 14** a) abstractie · b) decompositie · c) patroonherkenning
- 15** Abstractie, decompositie, patroonherkenning en het algoritme.
- 16** niet waar
- 17** De kleur van de huizen onderweg. Dat is abstractie: je laat weg wat er voor de weg niet toe doet.
- 18** Patroonherkenning. Je hergebruikt de oplossing die je voor het eerste niveau al gevonden hebt, in plaats van tien keer opnieuw te beginnen.
- 19** Als een probleem te groot is voor één keer, als verschillende delen los van elkaar werken, en als je met meerdere mensen samenwerkt.
- 20** apart werken aan de beweging, de punten en het geluid

4. Fouten en efficiëntie

- 21** een bug
- 22** debuggen
- 23** Je debugt het algoritme: je zoekt de stap waar het fout gaat en past die aan.
- 24** niet waar
- 25** niet waar
- 26** Er kunnen minder fouten in sluipen.
- 27** Abstractie, decompositie en patroonherkenning.

Een fout antwoord is het interessantste stuk van de bundel: vraag hoe je kind eraan kwam voor je zegt wat juist is. Vaak zit de vergissing één stap eerder dan je denkt.